Final Report: Project Semester

on

Development of Smart-Cart Application Using Java Spring Boot

Submitted by

Yash Bansal

102219073

(B.E., Electrical & Computer Engineering)

Under the Guidance of

Host Mentor
Confidential
Technology & Transformation Practice
Deloitte Touche Tohmatsu India LLP

Faculty Supervisor
Dr. Moon Inder Singh
Associate Professor
DEIE, TIET, Patiala

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

2026

Department of Electrical and Instrumentation Engineering
Thapar Institute of Engineering and Technology, Patiala
(Declared as Deemed-to-be-University u/s 3 of the UGC Act., 1956)
Post Bag No. 32, Patiala – 147004
Punjab (India)

I, Yash Bansal, hereby declare that this final report, titled “Development of Smart-Cart Application Using Java Spring Boot,” is my own account of the work I carried out as part of the Project Semester (UEE894) course, submitted towards the Bachelor of Engineering degree in Electrical & Computer Engineering at Thapar Institute of Engineering and Technology, Patiala.

The work described here was completed during my internship at Deloitte Touche Tohmatsu India LLP, under the day-to-day guidance of my Host Mentor in the Technology & Transformation practice, and under the academic supervision of Mr. Moon Inder Singh, Assistant Professor, Department of Electrical and Instrumentation Engineering, TIET, Patiala.

I further declare that no part of this report has been submitted previously, in full or in part, for a degree or diploma at this or any other institution.

Place: PatialaStudent Name: Yash Bansal
Date: _____Roll No.: 102219073

This is to certify that the above statement made by the student is correct to the best of my knowledge and belief.

Host MentorFaculty Supervisor
Deloitte Touche Tohmatsu India LLPDEIE, TIET, Patiala
ConfidentialDr. Moon Inder Singh
Technology & Transformation PracticeAssociate Professor

Acknowledgement

The past six months have been an incredible learning experience, and I would like to express my sincere gratitude to everyone who supported and guided me throughout this journey.

First of all, I would like to thank **Deloitte Touche Tohmatsu India LLP** for giving me the opportunity to intern in the **Technology & Transformation – Marketing and Commerce** practice. Being a part of a professional team and working on real projects gave me valuable exposure to the way enterprise software is developed and maintained. It allowed me to apply what I had learnt during my coursework while gaining practical experience that cannot be achieved in a classroom alone.

I am especially grateful to my **Host Mentor** and the entire team at Deloitte for their continuous guidance and encouragement. They were always approachable whenever I had questions and took the time to explain concepts, review my work, and suggest better ways of solving problems. Their feedback helped me improve not only my technical skills but also the way I approach software development, code quality, collaboration, and problem-solving. The experience of working with an experienced team has played a significant role in my professional growth.

I would also like to express my heartfelt thanks to **Dr. Moon Inder Singh**, Associate Professor, Department of Electrical and Instrumentation Engineering, TIET, Patiala, for supervising my Project Semester. His valuable suggestions, timely feedback, and constant encouragement helped me stay focused throughout the internship and prepare this report in a clear and organised manner.

I am equally thankful to **Thapar Institute of Engineering and Technology** for providing students with the opportunity to undertake the Project Semester (UEE894). The programme gave me the chance to experience the professional work environment while continuing my academic studies, helping me bridge the gap between theoretical concepts and their practical implementation.

Yash Bansal

Abstract

This report covers the six months I spent as a project semester intern at Deloitte Touche Tohmatsu India LLP, working within the Technology & Transformation (Marketing and Commerce) practice, starting January 23, 2026. The internship centered on enterprise software engineering and digital commerce platform work, with a fair amount of backend systems design mixed in.

The centerpiece of what I built was a Smart-Cart application in Java Spring Boot, developed for a confidential client engagement. Over the course of the project I touched most stages of the Software Development Life Cycle – gathering requirements, working through architecture decisions, writing the backend, hardening it against common security issues, testing it, and handling the pieces of deploymnt that fell to me. Security wasn't an afterthought here; I tried to build the application with an eye toward resisting the usual categories of web vulnerablities from the start, using secure coding habits rather than bolting security on at the end. All of this was version-controled through Git and GitHub, which meant getting comfortable with creating, managing, and merging a fair number of feature branches over time.

Beyond the Smart-Cart work, I also spent time building full-stack applications in React and Angular, worked on a payment backend that had to support recurring auto-pay logic, got some hands-on exposure to Apache Kafka and Elasticsearch, and completed Deloitte's structured training tracks in Generative AI, cybersecurity awareness, and professional ethics.

By the end of the internship, I had developed a strong understanding of backend development, REST API design, and PostgreSQL database management. More importantly, I gained first-hand experience of how software is designed, developed, and maintained in an enterprise environment. This report presents the objectives of the internship, the approach followed during the project, the technical work I carried out, the challenges I encountered, and the key lessons I learnt along the way. It also reflects on how this experience has contributed to my growth, both as a student and as an aspiring software engineer.

TABLE OF CONTENTS

List of Tables

List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CRUD	Create, Read, Update, Delete
ECE	Electrical & Computer Engineering
GenAI	Generative Artificial Intelligence
Git	Distributed Version Control System
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
JPA	Java Persistence API
JWT	JSON Web Token
Kafka	Apache Kafka (Distributed Event Streaming Platform)
LLM	Large Language Model
MVC	Model View Controller
ORM	Object Relational Mapping
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SQL	Structured Query Language
TIET	Thapar Institute of Engineering and Technology
UI	User Interface
UX	User Experience

Chapter 1

Introduction

Chapter Outline

This chapter lays out the industrial and organizational backdrop of the internship at Deloitte Touche Tohmatsu India LLP, describes the engineering problem I was brought in to help with, sets out the objectives I was working toward, and defines the scope of the six-month engagement.

1.1 Industry and Organizational Background

In recent years, organisations across different industries have increasingly adopted digital technologies to improve their business operations and enhance customer experience. As a result, software systems have become an essential part of managing day-to-day activities, handling customer interactions, and supporting online business. This has created a growing demand for scalable and reliable digital commerce platforms that can process large volumes of transactions, provide a seamless user experience, ensure secure payment processing, and support future growth. Consequently, skills in backend development, full-stack application development, and software architecture have become highly important in today's technology industry.

Deloitte Touche Tohmatsu India LLP is one of India's leading professional services firms, providing services in consulting, audit, risk advisory, financial advisory, and technology across multiple industries. Its **Technology & Transformation** practice helps organisations modernise their digital platforms by developing secure, scalable, and customer-focused software solutions. Within this practice, the **Marketing and Commerce** team specialises in building and maintaining enterprise web applications, e-commerce platforms, and digital commerce solutions that enable businesses to deliver better online experiences to their customers.

I did my internship within this Technology & Transformation – Marketing and Commerce practice, which gave me a good understanding to how enterprise software actually gets built, how client-facing solution delivery works, and what professional engineering standards look like inside a large consulting firm.

1.2 Problem Statement

Enterprise digital commerce platforms run into a cluster of engineering challenges that overlap and reinforce one another:

- Building backend systems that scale and stay maintainable under high transaction volumes.
- Designing application architectures that hold up against the common categories of cybersecurity vulnerabilities.
- Building frontend systems modular enough to stay responsive and consistent across the user experience.
- Designing payment architectures robust enough to handle recurring billing and its full lifecycle.
- Integrating APIs cleanly across a mix of internal and third-party services.
- Keeping structured data consistent through well-designed database schemas.
- Collaborating effectively across a distributed, remote development team using version control.

The core engineering problem I was assigned during the internship was building a Smart-Cart application in Java Spring Boot, for a client inside Deloitte India. That meant paying close attention to backend architecture, security, and making sure the work lined up with the SDLC standards the firm follows at the enterprise level.

Beyond Smart-Cart, the internship was also meant to close the gap between what I'd learned academically and how engineering actually works in industry – across full-stack development, distributed systems, and the softer skills of professional collaboration.

1.3 Internship Objectives

Going in, the internship was structured around six goals, laid out in the initial Goal Report:

- Goal 1 – Full-Stack Engineering Skills: Get real, hands-on experience building end-to-end applications with React, Angular, Java, and Spring Boot.
- Goal 2 – Backend Architecture Development: Build up backend engineering skill by working on payment systems, recurring transaction workflows, service-layer architecture, and REST APIs.
- Goal 3 – API Integration Capability: Understand how enterprise systems actually talk to each other – third-party APIs, internal APIs, authentication flows, and integration patterns.
- Goal 4 – Database Management: Build a working understanding of structured data systems using PostgreSQL, including query optimization and persistence design.
- Goal 5 – Scalable System Learning: Get exposure to distributed systems tools like Apache Kafka and Elasticsearch.
- Goal 6 – AI and Emerging Technology Exposure: Explore newer technologies, including Python, OpenCV, and Generative AI.

1.4 Scope of Work

The internship started on January 23, 2026, and ran fully remote for six months. Looking back, the work naturally broke down into three phases.

1.4.1 Learning and Onboarding Phase

The first few weeks went into onboarding – understanding what the project actually needed, reading through existing codebases, and picking up the internal development practices and collaboration norms that Deloitte India works with.

1.4.2 Development Phase

This was the bulk of the internship, and where most of the actual building happened: the Smart-Cart backend in Java Spring Boot, a full-stack e-commerce web application in React, enterprise module work in Angular, and the payment backend system. It also involved being

part of the structured SDLC process – requirements analysis, architecture design, code reviews, and managing everything through Git and GitHub.

1.4.3 Advanced Engineering and Consolidation Phase

The final phase of my internship focused on strengthening my understanding of backend development and learning how enterprise applications are designed for scalability and performance. During this period, I gained hands-on experience in debugging, testing, and maintaining production-level applications while also becoming familiar with deployment practices followed in an enterprise environment. I was also introduced to the practical applications of Generative AI in business and software development, which gave me a broader perspective on emerging technologies. In addition, I successfully completed Deloitte's mandatory training programmes covering professional ethics, cybersecurity awareness, data privacy, and organisational compliance, which helped me better understand the standards and responsibilities expected in a professional workplace.

Chapter 2

Review of Technologies and Concepts

Chapter Outline

This chapter walks through the core technologies and concepts I worked with during the internship, to give some technical grounding for the chapters that follow. It covers frontend technologies, backend systems, the Smart-Cart architecture, security practices, version control, database management, distributed systems, and the professional training I completed.

2.1 Frontend Development Technologies

On the frontend side, I worked with a fairly standard set of industry technologies to build responsive, modular interfaces:

- HTML and CSS: the foundational layer for structuring and styling web pages – still where everything else on the frontend ultimately rests.
- React: a component-based JavaScript library that's well suited to building reusable UI pieces. Its virtual DOM and declarative style make it a good fit for large applications where the UI updates frequently.
- Angular: a TypeScript-based framework maintained by Google, common in enterprise settings for its opinionated structure – dependency injection, routing, data binding, and form validation all come built in.

What ties these together is an emphasis on modular architecture and reusable, maintainable components – which is really just table stakes for enterprise-level development.

2.2 Backend Development: Java Spring Boot

Java Spring Boot was the backbone of my backend work throughout the internship, especially on the Smart-Cart application. Spring Boot is an open-source framework that takes a lot of the pain out of building production-ready Java applications, mainly through embedded server configuration, auto-configuration, and a large surrounding ecosystem.

Some of the specific Spring Boot capabilities I ended up relying on:

- RESTful API Design: building REST endpoints that follow standard HTTP verb conventions (GET, POST, PUT, DELETE) so frontend clients and backend services can talk to each other cleanly.
- Spring Security: setting up authentication, authorization, and protection against the usual suspects – CSRF, SQL injection, unauthorized access.
- Spring Data JPA: an ORM layer that cuts down on boilerplate when working with the database, keeping the persistence layer relatively clean.
- Service-Layer Architecture: keeping controller, service, and repository layers cleanly separated under an MVC pattern, which pays off in testability and modularity.
- Dependency Injection: using Spring's inversion-of-control container to manage object lifecycles and keep components loosely coupled.

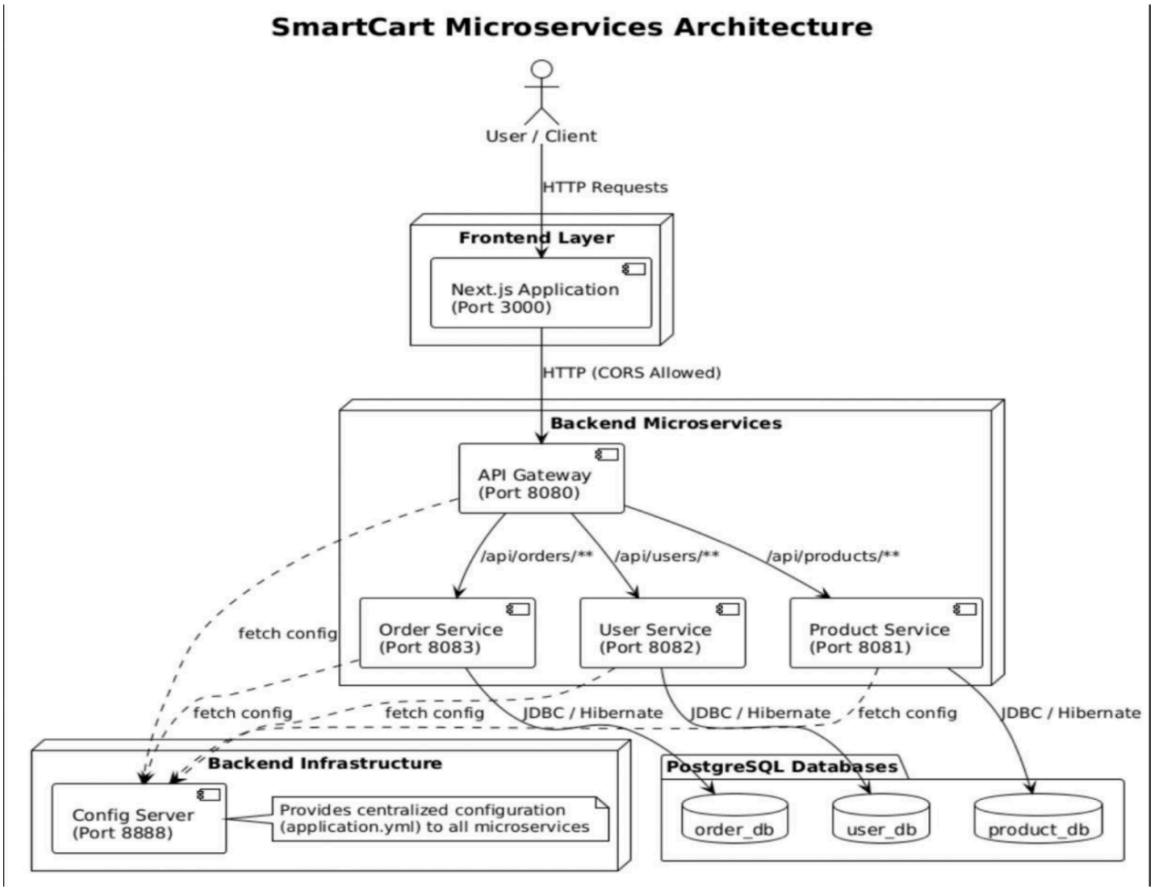
2.3 Smart-Cart Application Architecture

The Smart-Cart application was designed as a backend-heavy e-commerce solution meant to provide core shopping cart functionality for a client engagement. I tried to keep scalability, maintainability, and security in mind at every architectural decision, rather than treating them as concerns to revisit later.

The application ended up organized into four core layers:

- Presentation Layer: REST API endpoints exposed to frontend consumers, built to RESTful conventions so integration stays platform-agnostic.
- Business Logic Layer: service classes that handle the actual cart operations – adding items, removing them, adjusting quantities, calculating discounts, finalizing orders.
- Persistence Layer: repository interfaces managing the relational database through Spring Data JPA.
- Security Layer: authentication and authorization built with Spring Security, restricting access by role and closing off common web vulnerabilities.

Version control for Smart-Cart ran through Git and GitHub, with a structured branching approach – feature branches, pull requests, code review, and controlled merges into the main development branch.



2.4 Security and Cybersecurity Practices

Ensuring the security of the Smart-Cart application was an important part of the development process and was considered at every stage rather than being treated as a separate task. Several security measures were implemented to make the application more reliable and resistant to common vulnerabilities.

- **Input Validation and Sanitisation:** All data received through the API was validated and sanitised before processing to reduce the risk of attacks such as SQL injection and Cross-Site Scripting (XSS).
- **Authentication and Authorisation:** Role-based access control (RBAC) was implemented to ensure that users could only access features and perform actions permitted by their assigned roles. This helped protect sensitive operations such as managing shopping carts and placing orders.

- **Secure API Development:** REST APIs were designed by following secure development practices, including proper error handling, controlled data exposure, and validation of incoming requests to prevent unauthorised access and information leakage.
- **Dependency Management:** Third-party libraries and project dependencies were regularly reviewed and updated to minimise the risk of introducing known security vulnerabilities into the application.

2.5 Version Control with Git and GitHub

Version control was an integral part of the development process throughout my internship. The team used **Git** for distributed version control and **GitHub** to host and manage the project repositories. Working in this environment helped me understand the importance of maintaining a structured workflow when multiple developers contribute to the same codebase. Some of the key practices I followed are described below:

1. Branch Management**:** A separate feature branch was created for every assigned task or enhancement. This ensured that ongoing development remained isolated from the main branch, reducing the risk of affecting stable code.
2. Pull Requests and Code Reviews**:** Once a feature was completed, a pull request was raised for review. Team members carefully reviewed the changes, suggested improvements where required, and approved the code before it was merged. This process helped improve code quality and encouraged knowledge sharing within the team.
3. Merge Management**:** Feature branches were merged into the main branch only after successful code review and approval. Whenever merge conflicts occurred, they were resolved carefully to maintain consistency and avoid introducing errors into the codebase.

2.6 Database Management with PostgreSQL

PostgreSQL was the database I worked with most throughout the internship. My main activities here included:

- Schema Design: designing normalized schemas to model entities like users, products, cart items, and orders.

- CRUD Operations: implementing create, read, update, and delete operations through Spring Data JPA repositories.
- Query Optimization: writing and tuning SQL queries where data retrieval was getting slow.
- Data Persistence Design: thinking through how data should be managed over its lifecycle, to keep things consistent and reliable.

2.7 Distributed Systems and Scalable Technologies

The later stage of the internship introduced me to technologies commonly used in distributed and large-scale enterprise systems.

Apache Kafka: Apache Kafka is an open-source event streaming platform used for building real-time data pipelines and event-driven applications. Its publish–subscribe architecture enables reliable communication between different services.

Elasticsearch**:** Elasticsearch is a distributed search and analytics engine used for full-text search, log analysis, and real-time data indexing. It is widely used in e-commerce applications to provide fast and efficient product search.

Although I did not implement these technologies directly in production, learning about them helped me understand how scalable enterprise applications are designed and managed.

2.8 Generative AI and Professional Training

On my own initiative, I worked through a structured 17-hour Generative AI course covering:

- Large Language Model concepts and the basics of how they're architectured.
- Prompt engineering strategies for getting more out of AI-assisted tools.
- AI-assisted development workflows and where they actually fit in enterprise software engineering.

Alongside that, I completed several of Deloitte's internal training programs, which together reinforced what professional, responsible engineering practice is supposed to look like:

- Ethics and Professional Conduct: Deloitte's code of professional ethics and what it means for client engagements.
- Cybersecurity Awareness: practical grounding in common threats and the organization's best practices for protecting data.
- Organizational Compliance Training: getting familiar with internal policies and the regulatory requirements that govern professional behavior at the firm.

Chapter 3

Methodology

Chapter Outline

This chapter describes how I approached the internship's work – my participation in the SDLC, the development workflow I followed for Smart-Cart and other assignments, how collaboration and version control were handled, and the testing and maintenance strategy I used.

3.1 SDLC Participation

One of the more defining parts of the internship was getting real ownership over major phases of the Software Development Life Cycle, rather than just watching from the sidelines. The overall process leaned Agile – iterative development, constant feedback, incremental delivery. Here's how the main phases played out.

3.1.1 Requirements Understanding and Analysis

Every development task began with understanding the requirements and expected functionality. This involved reviewing user stories and project requirements, discussing any queries with senior developers or my Host Mentor, and breaking the work into smaller, manageable tasks. For the Smart-Cart application, this included understanding the expected cart functionality, security requirements, and how the module would integrate with the rest of the system.

3.1.2 Application Architecture and Design

Once the requirements were understood, the application architecture was planned for the Smart-Cart module. This involved adopting the Controller–Service–Repository architecture, designing REST API endpoints, defining database models, and planning the application's security. Throughout this phase, emphasis was placed on developing a solution that was modular, maintainable, scalable, and easy to test.

3.1.3 Backend Development

This was the main development stretch – implementing the requirements in Java Spring Boot. REST endpoints came together, business logic sat in service classses, and persistence ran through Spring Data JPA repositories connected to PostgreeSQL. Security got built in at both the API and service layers, not tacked on afterward.

3.1.4 Testing and Debugging

Unit and integration testing ran throughout, not just at the end, to check that features actually worked as intended. Debugging covered both logical bugs caught by tests and issues that came up during peer code review.

3.1.5 Deployment-Related Activities

During the deployment phase, I worked on preparing the application for integration into the development environment. This included managing configuration properties for different environments and ensuring that the application started and ran consistently across deployments.

Table 3.1 summarizes how these phases mapped to specific activities and the tools involved.

SDLC Phase	Key Activities	Technologies/Tools
RequirementsAnalysis	User story review, task decomposition, API contract definition	Confluence, team meetings
Architecture Design	Layer design, data model planning, security framework selection	UML diagrams, Spring Boot
BackendDevelopment	API development, business logic, persistence layer, security	Java, Spring Boot, PostgreSQL
Testing & Debugging	Unit tests, integration tests, defect resolution, code reviews	JUnit, debugging tools
Version Control	Branch management, pull requests, code reviews, merges	Git, GitHub
DeploymentActivities	Configuration management, environment setup, runtime validation	Spring Boot profiles, command-line tools

3.2 Development Workflow

Day to day, the workflow I settled into for each task looked roughly like this:

- Requirement Understanding: reading through the task, clarifying anything unclear with the mentor or team, and roughly planning out the implementation before writing code.
- Implementation: writing code that's clean and reasonably documented, sticking to the project's coding standards and architecture conventions.
- Self-Review: going back over my own work for correctness and completeness before sending it out for review.
- Code Review and Iteration: taking part in peer reviews, folding in the feedback, and refining the implementation accordingly.
- Integration: merging approved changes into the development branch through the pull request process on GitHub.

3.3 Collaboration and Version Control Practices

Working remotely, collaboration leaned heavily on structured version control and clear communication. Git branches got created per feature or task, with names descriptive enough to tell you what the branch was actually for. Finished branches went up as pull requests on GitHub, where teammates could comment, request changes, or approve. Once approved, they'd get merged into the integration branch.

This kept the main codebase stable, since nothing landed there without going through review first. Managing several branches at once – creating, reviewing, merging – became a routine part of the work rather than an exception.

3.4 Testing, Debugging, and Maintenance

Testing was carried out at both the individual component level and the integrated application level to ensure that all features worked as expected. Whenever issues were encountered, they were identified, analysed, and resolved through a structured debugging process. Maintenance activities included implementing requirement changes, improving code quality through refactoring, and fixing bugs identified during integration and testing.

Chapter 4

Work Done and Technical Contributions

Chapter Outline

Chapter Outline

This chapter presents the technical work carried out during the internship, organised according to the different projects and areas of responsibility. It includes the development of the Smart-Cart application, the payment backend system, the e-comdmerce web application, Angularr module development, exposure to distributed systems, and the AI and professional training programmes completed during the internship.

4.1 Smart-Cart Application Development

Building the Smart-Cart application in Java Spring Boot was, without question, the main technical contribution I made during the internship – a client-facing backend engagement within Deloitte's Technology & Transformation practice that touched nearly every stage of the SDLC, from initial requirements through to deployment.

4.1.1 Requirements Understanding and Feature Design

The development of the Smart-Cart application began with analysing the functional requirementss of the system. This involved identifying the key entities, such as userrs, products, cart items, and orders, along with the core functionalities, including adding products to the cart, updating quantities, removing items, and placing orders. Alongside these functional requirements, factors such as performance, securityy, and maintainability were also considered. Before implementation, the REST API endpoints, request and response formats, and appropriate HTTP methods were planned to ensure a well-structured and consistent backend design.

4.1.2 Backend Development and API Implementation

The backend of the Smart-Cart application was developed using **Java Spring Boot** following the **Model–View–Controller (MVC)** architecture. RESTful APIs were created to handle all cart-related operations and enable between the frontend and backend. The **Controller** layer managed incoming requests and responses, the **Service** layer contained the business logic, and the **Repository** layer handled database operations using **Spring Data JPA** with **PostgreSQL** as the database.

Table 4.1 lays out the core technology stack behind the Smart-Cart application.

Component	Technology	Purpose
Backend Framework	Java Spring Boot	API and business logic
ORM / Persistence	Spring Data JPA	Database interaction
Database	PostgreSQL	Relational data storage
Security	Spring Security	Auth & vulnerability protection
API Style	RESTful (HTTP/JSON)	Client-server communication
Version Control	Git / GitHub	Source management & collaboration

4.1.3 Security Implementation

Security was an important consideration throughout the development of the Smart-Cart application. Using Spring Security, authentiication was implemented to ensure that only authorised users could access cart-related features. Role-based authorisation was used to restrict sensitive operations based on user permissions. Input validation was performed at the API level to prevent common attacks such as SQL injectionn and invalid data submissions. Additionally, API responses were carefully designed to avoid exposing sensitive system information, making the application more secure and aligned with enterprise development standards.

4.1.4 Version Control and Branch Management

Smart-Cart's version control ran through Git on GitHub, with a separate feature branch for each task. Once a feature was done, I'd open a pull request for review, work through the feedback,

and merge once it was approved. Managing several branches at the same time – and resolving the occasional merge conflict – became routine rather than something to dread.

4.1.5 Testing, Debugging, and Maintenance

I wrote unit tests to check individual service and repository components, and integration tests to confirm the API endpoints behaved correctly or not. When testing or code review turned up a defect, I'd trace it to the root cause and fix it there rather than the symptoms. Maintenance meant refactoring where the code had gotten messy and updating implementations when the requirements changed.

4.2 Payment Backend System

The payment backend system was developed using Java Spring Boot to manage the complete payment lifecycle for a digital commerce platform. The application was designed to support both one-time payments and recurring subscription payments while ensuring secure and reliable transaction processing.

The major contributions made during this project include:

- Transaction Management: Implemented the complete payment flow, including payment initiation, processing, successful transasction handling, and failure management.
- Recurring Payments: Developed the backend logic to support subscription-based payments, enabling automatic billing at predefined intervals.
- Service Layer Design: Followed a layered architecture by separating business logic from the API layer, resulting in cleaner, more maintainable, and easily testable code.
- REST API Development: Developed RESTful APIs to enable communication between the frontend application and exterrnnal payment gateway services.
- Input Validation and Error Handling: Implemented input validation and structured exception handling to ensure reliablle system behaviour and appropriate responses for invalid requests.

4.3 E-Commerce Web Application

I also worked on a full-stack e-commerce web application for a client, with the frontend built in React around a component-based architecture aimed at modularity and reuse.

My contributions here included:

- Responsive UI Design: building UI components that adapt across screen sizes, so the experience holds up on both desktop and mobile.
- Product Listing Components: dynamic product listing pages that render variable catalogs pulled from backend APIs.
- User Interaction Workflows: the actual user-facing flows – browsing products, interacting with the cart, starting checkout.
- Modular React Architecture: organizing the frontend into components that are reusable and independently maintainable, in line with the project's frontend standards.

4.4 Angular Enterprise Module Development

Separately, I used Angular to build structured enterprise-grade modules inside existing enterprise web systems – my first real exposure to a large-scale enterprise frontend framework in a professional setting.

This work included:

- Component Creation: building Angular components to handle specific UI behaviors and data presentation.
- Data Binding Implementation: using Angular's two-way data binding to keep UI state in sync with the underlying data model.
- Routing Configuration: setting up Angular's router to manage navigation between views in the enterprise application.
- UI Validation Workflows: form validation through Angular's reactive forms module, to keep data integrity intact before submission.

4.5 Distributed Systems Exposure

The internship also gave me structured exposure to the distributed systems technologies behind enterprise-scale digital platforms:

- Apache Kafka: I studied Kafka's event-driven model, its producer-consumer architecture, and topic-based routing, which helped me understand how it decouples microservices and enables asynchronous communication – useful architectural insight for high-volume commerce systems.
- Elasticsearch: I explored its indexing model, query DSL, and how it typically integrates into backend systems, particularly around product search optimization and real-time analytics in a commerce platform.

4.6 AI and Professional Training Programs

Alongside the hands-on development work, I completed the following structured learning programs:

- Generative AI Course (17 hours): LLM fundamentals, prompt engineering techniques, and practical AI-assisted development workflows relevant to enterprise software engineering.
- Ethics and Professional Conduct Training: professional responsibilities, client confidentiality, and conduct standards inside a consulting organization.
- Cybersecurity Awareness Training: common threat vectors, organizational security protocols, and best practices for protecting data and systems.
- Organizational Compliance Training: Deloitte's internal policies, regulatory requirements, and procedural guidelines.

Chapter 5

Progress Assessment and Key Learnings

Chapter Outline

This chapter checks progress against the original internship goals, sums up the technical and professional skills I picked up, and reflects on what actually got in the way and how I worked through it over the six months.

5.1 Goal-Wise Progress Assessment

Table 5.1 gives a structured look at where I landed against each of the six goals set at the start of the internship.

Goal	Status	Key Achievements
Full-Stack Engineering Skills	Achieved	Built e-commerce app (React), enterprise modules (Angular), backend (Spring Boot)
Backend Architecture Development	Achieved	Developed Smart-Cart app, payment backend with lifecycle management and auto-pay
API Integration Capability	Achieved	REST API development, service integration, authentication workflows
Database Management	Achieved	PostgreSQL schema design, CRUD operations, query optimization using JPA
Scalable System Learning	Achieved	Conceptual and practical exposure to Apache Kafka and Elasticsearch
AI and Emerging Technology	Achieved	Completed 17-hour GenAI course; explored AI-assisted development workflows

5.2 Technical Skills Acquired

Table 5.2 summarizes the technical skills I built up, grouped by domain.

Domain	Skills Acquired
Backend Development	Spring Boot REST API design, service-layer architecture, JPA-based persistence, Spring Security
Frontend Development	React component design, Angular module development, responsive UI, data binding, routing
Security Engineering	Input validation, authentication & authorization, secure API design, cybersecurity awareness

Domain	Skills Acquired
Database Management	PostgreSQL schema design, CRUD operations, SQL query optimization, JPA repositories
Version Control	Git branching strategy, pull requests, code reviews, merge management, conflict resolution
SDLC Practices	Requirements analysis, architecture design, iterative development, testing, deployment
Distributed Systems	Apache Kafka event streaming concepts, Elasticsearch indexing and search
Artificial Intelligence	LLM fundamentals, prompt engineering, AI-assisted development workflows

5.3 Challenges Faced and Overcome

The internship threw up a fair number of technical and professional challenges, and working through them probably taught me as much as the actual project work did.

5.3.1 Transition from Academic to Enterprise Workflows

Moving from tidy academic assignments to open-ended enterprise tasks took some real adjustment. Academic projects come with a well-defined problem statement; real engineering work comes with ambiguous requirementts, specifications that shift under you, and dependencies on other people's schedules. I got through this mostly by getting into the habit of asking clarifying questions early and delivering in smaller increments rather than waiting to have everything figured out.

5.3.2 Navigating Large Enterprise Codebases

Getting comfortable in large existing codebases – especially during the e-commerce and Angular work – meant reading code systematically, going through documentation, and slowly building up familiarity rather than expecting to understand it all at once. Learning to work inside an established structure without breaking things that already worked took time, but it got noticeably easier over the course of the internship.

5.3.3 Managing Multiple Technologies Simultaneously

Keeping track of Java Spring Boot, React, Angular, PostgreSQL, Git, Kafka, and Elasticsearch all at once meant a lot of context-switching. I managed it mostly by prioritizing based on what mattered most for the project at hand, and leaning on Deloitte's internal training resources to fill in gaps rather than trying to learn everything from scratch on my own.

5.3.4 Remote Collaboration

Being fully remote meant I had to be more deliberate about using collaboration tools, writing clearly, and proactively sharing status updates to stay aligned with the team. These weren't things I did well from day one – they got better through practice.

5.3.5 Security Implementation Complexity

Building out security for Smart-Cart meant understanding not just the Spring Security framework itself, but the broader landscape of common web vulnerabilities and how to actually mitigate them. Working through Spring Security's documentation, the cybersecurity training, and iterating on feedback from my mentor is what eventually got this to a solid place.

5.4 Professional Development

The internship also contributed significantly to my professional development beyond technical skills. Working in Deloitte's Technology & Transformation practice helped me understand the expectations and responsibilities of working in a professional software development environment. Some of the key learnings include:

- Enterprise Development Practices: Gained an understanding of the standards, workflows, and quality expectations followed in large-scale software development projects.
- Client-Oriented Approach: Learnt the importance of developing secure, reliable, and scalable solutions that meet client requirements and business objectives.
- Team Collaboration: Improved my ability to work effectively within a team through structured version control, code reviews, and regular communication.
- Professional Ethics and Compliance: Developed a better understanding of professional ethics, cybersecurity, data privacy, and organisational compliance through Deloitte's training programmes

Chapter 6

Conclusions and Future Work

6.1 Conclusions

Looking back, my six-month Project Semester internship at **Deloitte Touche Tohmatsu India LLP** in the **Technology & Transformation (Marketing and Commerce)** practice has been one of the most valuable learning experiences of my academic journey. It gave me the opportunity to work on real-world projects, apply my classroom knowledge, and understand how software is developed in an enterprise environment. Throughout the internship, I was able to achieve the objectives I had set and contribute to backend development, payment systems, and enterprise web applications.

A major part of my internship involved developing the **Smart-Cart** application using **Java Spring Boot**. This project exposed me to different stages of the Software Development Life Cycle, including requirement analysis, application design, development, testing, and deployment. It also strengthened my understanding of backend development, REST API design, PostgreSQL, Git, and GitHub. Along the way, I learnt the importance of building secure applications by implementing authentication, authorisation, and input validation using Spring Security.

Apart from the Smart-Cart project, I worked on a Spring Boot-based payment backend and contributed to enterprise web applications developed using React and Angular. I also gained basic exposure to technologies such as Apache Kafka and Elasticsearch, along with completing Deloitte's training programmes on Generative AI, cybersecurity, professional ethics, and organisational compliance.

The internship also presented several challenges, such as working with large codebases, learning new technologies, and adapting to enterprise development practices. With regular guidance from my mentor and support from the team, I was able to overcome these challenges and improve both my technical and professional skills.

Overall, this internship helped bridge the gap between academic learning and industry practice. It strengthened my technical knowledge, improved my problem-solving abilities, and gave me the confidence to begin my career in software engineering with a solid foundation.

6.2 Future Work

Building on the knowledge and experience gained during this internship, I would like to continue developing my skills in the following areas:

- Advanced Spring Boot: Explore advanced concepts such as reactive programming, microservices, and cloud-nativve application development.
- Microservices and Containerisation: Gain practical experience with Docker and Kubernete for deploying and managing scalable applications.
- Application Security: Develop a deeper understanding of OAutth 2.0, OpenID Connect (OIDC), and secure software development practices based on OWASP guidelines.
- DevOps Practices: Learn to design and implement CI/CD pipelines for automated testing, building, and deployment of applications.
- Distributed Technologies: Gain hands-on experience with Apache Kafkaa and Elasticsearch by integrating them into enterprise applications.
- AI in Backend Development: Explore the integration of Generative AI and Large Language Models (LLMs) into backend systems to build intellige application features.
- Smart-Cart Enhancements: Extend the Smart-Cart application by adding features such as product recommendations, real-time inventory management, multi-currency support, and enhanced analytics.

References

[1] G. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.

[2] R. C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2008.

[3] Spring Framework Documentation – Spring Boot Reference Guide, Pivotal Software, 2024. [Online]. Available: https://docs.spring.io/spring-boot/docs/current/reference/html/

[4] Spring Security Reference Documentation, Pivotal Software, 2024. [Online]. Available: https://docs.spring.io/spring-security/reference/

[5] PostgreSQL Documentation, The PostgreSQL Global Development Group, 2024. [Online]. Available: https://www.postgresql.org/docs/

[6] Apache Kafka Documentation, Apache Software Foundation, 2024. [Online]. Available: https://kafka.apache.org/documentation/

[7] Elasticsearch Reference Documentation, Elastic N.V., 2024. [Online]. Available: https://www.elastic.co/guide/en/elasticsearch/reference/current/

[8] React Documentation, Meta Open Source, 2024. [Online]. Available: https://react.dev/

[9] Angular Documentation, Google LLC, 2024. [Online]. Available: https://angular.dev/

[10] OpenAPI Specification, SmartBear Software, 2024. [Online]. Available: https://swagger.io/specification/

[11] OWASP Top Ten, Open Web Application Security Project, 2021. [Online]. Available: https://owasp.org/www-project-top-ten/

[12] Git Reference Documentation, Software Freedom Conservancy, 2024. [Online]. Available: https://git-scm.com/docs

[13] Deloitte India – Technology & Transformation Practice Overview, Deloitte Touche Tohmatsu India LLP, 2024. [Online]. Available: https://www2.deloitte.com/in/en.html

Appendix A – Project Timeline

The table below lays out the planned project timeline for the six-month internship, as set out in the initial Goal Report and tracked through the midway point.

Task / Activity	Month 1	Month 2	Month 3	Month 4	Month 5	Month 6
Onboarding & Orientation	✓					
Smart-Cart Backend Development		✓	✓	✓		
Frontend Development (React, Angular)			✓	✓	✓	
Payment Backend System				✓	✓	
API Integration Work		✓	✓	✓	✓	
Database Work (PostgreSQL)			✓	✓	✓	
Distributed Systems Learning				✓	✓	✓
AI & Professional Training			✓	✓	✓	
Final Report Preparation					✓	✓

Appendix B – List of Key Tools and Technologies

The table below summarizes the key tools and technologies I used across the internship.

Tool / Technology	Category	Usage During Internship
Java	Programming Language	Primary backend development language
Spring Boot	Backend Framework	Smart-Cart app development, payment backend, REST APIs
Spring Security	Security Framework	Authentication, authorization, vulnerability mitigation
Spring Data JPA	ORM Library	Database entity management and repository layer
PostgreSQL	Database	Relational data storage for application entities
React	Frontend Framework	E-commerce web application UI development
Angular	Frontend Framework	Enterprise web module development
Git	Version Control	Source code management, branching, merging
GitHub	Repository Hosting	Remote repository, pull requests, code reviews
Apache Kafka	Distributed Systems	Event streaming concepts and enterprise messaging study
Elasticsearch	Search & Analytics	Search optimization and distributed indexing study
Python / OpenCV	AI & Vision	Emerging technology exploration
HTML / CSS	Frontend	Structured layout and responsive styling

Plagiarism Report

The plagiarism report must be signed by the faculty member. Only the first page of the report should be included in the final project submission.

[Attach plagiarism report here]